

AI AUGMENTED BACKEND DEVELOPMENT -AI FAQ ASSISTANT API

Introduction

Team Members

- **Varshneleshan V**
- **Yogeshwari N**
- **Aishwarya**
- **Vijayalakshmi**
- **Vaishnavi T.K**

1.1 Background

FAQ systems are widely used by educational institutions, businesses, online services and support teams. Traditional FAQ pages require users to browse categories or search for exact words. As the number of entries grows, finding the right answer becomes slower, and users often give up and contact support directly.

An AI FAQ Assistant allows users to ask questions naturally. The backend processes the query and attempts to locate a relevant stored answer, even when the wording differs from the stored question. This reduces the effort required to find information and improves the overall user experience.

1.2 Scope of the Project

The project focuses on the API layer so that different client applications can use the same question-answer service. The scope covers the following areas:

- A REST API for asking questions (**/api/ask**) and for managing FAQ records (create, read, update and delete).
- Storage of FAQ entries with category, tags, timestamps and embedding information.
- Semantic, similarity-based retrieval with a configurable relevance threshold and fallback response.
- Authentication for administrative operations, request validation, rate limiting and predictable error responses.
- Testing of every endpoint using Postman, with a plan for automated tests.

A graphical user interface is outside the scope of this project. Any web, mobile or desktop client can be developed separately by following the API contract described in this document.

1.3 Motivation

Repeated questions consume support time and delay responses for users who genuinely need human help. A reusable backend service that answers common questions instantly is valuable to almost any organisation. The project also provides practical experience in REST design, database modelling, security practice and the integration of AI services into a conventional backend.

2. Problem Statement

Traditional FAQ systems often depend on manual browsing or simple keyword matching. Users may understand the information they need but use different wording from the stored FAQ question. For example, a user may type “I forgot my login secret” while the FAQ says “How can I reset my password?”. A keyword search may return nothing even though a suitable answer exists.

Maintaining a large FAQ collection can also become difficult. Administrators need reliable operations for adding, updating and deleting FAQ entries, and the changes must be reflected in the search index without manual rework.

The proposed system addresses these issues through an AI-assisted retrieval service exposed through a REST API. The main problems identified are:

- **Manual searching:** users must browse long lists of questions to find an answer.
- **Exact-keyword dependency:** differences in wording lead to failed searches.

- **Repeated support questions:** staff answer the same questions many times.
- **Maintenance difficulties:** there is no centralised, consistent way to manage FAQ content.

3. Objectives

The primary objective is to develop a reliable API that accepts natural-language questions and returns relevant FAQ answers. The system should be simple enough for an academic project while following standard software engineering practices. The specific objectives are:

- Create REST endpoints for FAQ management and question answering.
- Implement the `/api/ask` endpoint that accepts a question and returns the best matching answer.
- Store FAQ data, including categories, tags and embeddings, in a database.
- Use semantic or similarity-based retrieval instead of exact keyword matching.
- Return structured JSON responses for all successful and failed requests.
- Validate requests, protect administrative operations and handle errors consistently.
- Design the system so that future web and mobile clients can be integrated without changes to the backend.

4. Existing System

An existing manual FAQ system normally presents a collection of questions and answers on a webpage. Users browse categories or use a basic search field. Keyword-based systems can fail when the user's wording differs from the stored question. Organisations may also depend on human support staff for repeated questions, increasing response time and workload.

The limitations of the existing approach are summarised in Table 5.1.

Table 4.1: Limitations of the existing FAQ system

Aspect	Limitation	Impact
Manual browsing	Users scroll through categories and long pages	Time-consuming; information is often missed
Keyword search	Matches only exact or near-exact words	Relevant answers are not found when wording differs
Human intervention	Support staff answer repeated questions	Higher workload and slower response time
Language understanding	No interpretation of meaning or intent	Poor experience for natural-language questions
Content management	Manual editing of pages	Inconsistent and outdated information

5. Proposed System

The proposed system provides an AI-assisted FAQ API. A user sends a question to the `/api/ask` endpoint. The backend validates the request, processes the question, searches the FAQ knowledge base and selects the most relevant answer. Semantic retrieval represents both stored questions and incoming queries as vectors; similar meanings are then identified using a similarity measure.

5.1 Processing Workflow

1. The client sends a JSON request containing the user's question.
2. The API validates the request body and applies authentication and rate-limit rules where required.
3. The text processing module cleans and normalises the question.
4. The semantic search module converts the question into an embedding vector.
5. The vector is compared with the stored FAQ embeddings using a similarity measure.
6. If the best score meets the relevance threshold, the stored answer is returned; otherwise a fallback message is returned.
7. The API sends a structured JSON response containing the answer and confidence value.

5.2 Key Features

- Natural-language input instead of exact keywords.
- Semantic retrieval based on meaning.
- REST communication that works with any HTTP client.
- Structured JSON output with a confidence value.
- Full CRUD operations for managing FAQ records.

6. AI-Augmented Development Approach

The title of this project refers to AI-augmented backend development. The term has two meanings here. First, the backend itself is augmented with AI: an embedding or language-model service gives the API the ability to understand questions by meaning. Second, the development process is augmented with AI tools that assist the developer during design, implementation, testing and documentation.

6.1 Where AI Assists the Development Process

AI assistants are used as productivity aids, not as decision makers. Table 7.1 lists the project phases in which they are used and the responsibility that always remains with the developer.

Table 6.1: AI assistance and developer responsibility by phase

Phase	AI assistance	Developer responsibility
Requirements	Brainstorming edge cases and missing requirements	Decide final scope and priorities
Design	Suggesting endpoint structures and database fields	Approve the design and check it against REST principles
Implementation	Drafting boilerplate, routes, validation and helper functions	Read, understand and correct every generated line
Testing	Proposing test cases and sample inputs	Run the tests and confirm expected results
Documentation	Drafting and polishing text and comments	Verify technical accuracy

6.2 Guidelines Followed

- **Human review:** no generated code is accepted without being read, understood and tested.
- **Security first:** credentials, API keys and real user data are never pasted into AI tools or committed to source control.
- **Small, verifiable steps:** features are built and tested one endpoint at a time.
- **Version control:** every change is committed to Git so it can be inspected or reverted.
- **Accountability:** the developer remains responsible for the correctness, licensing and security of the final code.

7. System Requirements

A typical implementation can use Node.js and Express for the backend, MongoDB or MySQL for storage, Postman for API testing, and an embedding or NLP service for semantic retrieval. The suggested technologies are listed in Table 8.1.

Table 7.1: Suggested software requirements

Requirement	Suggested Technology
Operating System	Windows / Linux
Editor	Visual Studio Code
Backend	Node.js + Express
Database	MongoDB / MySQL
API Testing	Postman
Version Control	Git
AI / NLP	Embedding or language model service

7.1 Hardware Requirements

The application is lightweight and can be developed on an ordinary computer. A processor with two or more cores, at least 8 GB of RAM and a few gigabytes of free disk space are sufficient

for development and testing. Production capacity depends on the number of FAQ entries and the expected number of requests.

7.2 Configuration

Configuration values are supplied through environment variables so that no secret is stored in source code. The main variables are shown below.

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/faq_assistant
ADMIN_API_KEY=<long-random-secret>
EMBEDDING_API_KEY=<provider-key>
SIMILARITY_THRESHOLD=0.75
```

8. System Architecture

The system follows a layered architecture. The client sends HTTP requests to the API. The API validates the request and passes it to the service layer. The service layer performs question processing and retrieval, and the data layer stores FAQ records.

Architecture flow: Client Application → REST API → Validation → AI / Semantic Retrieval → FAQ Database → JSON Response.

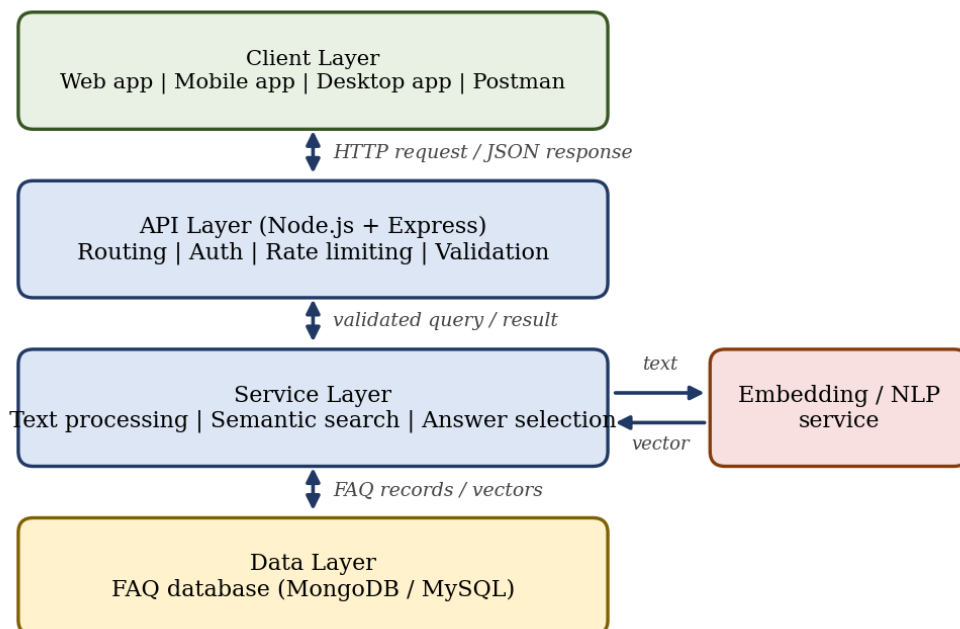


Figure 8.1: Layered architecture of the AI FAQ Assistant API

Separating these layers makes the application easier to test, maintain and extend. For example, the embedding provider can be replaced by changing only the service layer, and the database can be changed without modifying the route handlers.

8.1 Suggested Project Structure

```
faq-assistant-api/
  src/
    app.js           Express setup and middleware
    server.js        Server start-up
    routes/
      faq.routes.js  CRUD endpoints for FAQs
      ask.routes.js  POST /api/ask
    middleware/
      auth.js        Admin authentication
      validate.js    Request validation
      errorHandler.js Central error handling
    services/
      embedding.service.js Calls the embedding provider
      retrieval.service.js Similarity search and answer selection
    models/
      Faq.js         Database schema
  tests/
  .env.example
  package.json
```

9. Functional Modules

The project is divided into modules, with each module responsible for one part of the system. Table 10.1 describes the purpose of each module.

Table 9.1: Functional modules

Module	Responsibility
User Query Module	Receives the question from the client through the /api/ask endpoint.
Validation Module	Checks that required fields exist, have the correct type and stay within length limits.
Text Processing Module	Cleans the text by trimming spaces, normalising case and removing unnecessary characters.
Semantic Search Module	Generates the query embedding and calculates similarity against stored FAQ embeddings.
Answer Module	Applies the relevance threshold and builds the JSON response or fallback message.
FAQ Management Module	Provides create, read, update and delete operations for FAQ records.
Authentication Module	Protects administrative operations using an API key or bearer token.
Error Handling Module	Converts errors into consistent JSON responses with suitable status codes.

When an FAQ is created or updated, the FAQ Management Module calls the Semantic Search Module to generate a new embedding for the question so that the search index always matches the stored content.

10. Database Design

The FAQ database stores the information required to answer user questions. A basic record contains an identifier, question, answer, category, tags, embedding information and timestamps. The fields are described in Table 10.1.

Table 10.1: FAQ record structure

Field	Type	Description
id	ObjectId / Integer	Unique identifier of the FAQ record
question	String (required)	The stored FAQ question
answer	String (required)	The answer returned to users
category	String	Grouping such as Account, Billing or Admissions
tags	Array of Strings	Keywords that describe the entry
embedding	Array of Numbers	Vector representation of the question
isActive	Boolean	Allows an entry to be hidden without deleting it
createdAt / updatedAt	Date	Timestamps maintained automatically

10.1 Example Record

```
{
  "id": 101,
  "question": "How can I reset my password?",
  "answer": "Open account settings and select the password reset option.",
  "category": "Account",
  "tags": ["password", "account", "reset"],
  "embedding": [0.012, -0.087, 0.143, "..."],
  "createdAt": "2026-01-10T09:30:00Z",
  "updatedAt": "2026-01-10T09:30:00Z"
}
```

An optional administrators collection can store a username, a password hash (never the plain password) and a role when token-based authentication is used instead of a single API key. For a small knowledge base the embeddings are compared in application memory; for larger collections a database with native vector search can be used.

11. API Design

The API follows REST principles. GET, POST, PUT and DELETE methods are used for resource operations, while JSON is used for requests and responses. Read operations and the assistant endpoint are public, and operations that change the knowledge base require administrator authentication.

Table 11.1: API endpoints

Method	Endpoint	Purpose	Access
GET	/	API status	Public
GET	/api/faqs	List FAQs	Public
GET	/api/faqs/:id	Get one FAQ	Public
POST	/api/faqs	Create FAQ	Admin
PUT	/api/faqs/:id	Update FAQ	Admin
DELETE	/api/faqs/:id	Delete FAQ	Admin
POST	/api/ask	Ask assistant	Public (rate limited)

Every response uses the same envelope: a **success** flag followed by either the requested data or an **error** object. This consistency allows client applications to handle results with a single piece of logic.

12. API Request and Response Examples

12.1 Asking a Question

POST /api/ask receives a JSON question and returns the best matching answer.

Request:

```
POST /api/ask
Content-Type: application/json

{ "question": "How can I reset my password?" }
```

Response (200 OK):

```
{
  "success": true,
  "question": "How can I reset my password?",
  "answer": "You can reset your password from account settings.",
  "confidence": 0.94
}
```

The response includes a confidence value because the implementation calculates a similarity score. Clients may use it, for example, to display a “was this helpful?” prompt only for lower-confidence answers.

12.2 Question With No Relevant Answer

When the best similarity score is below the configured threshold, the API returns a fallback message instead of an unrelated answer.

```
{
  "success": true,
  "matched": false,
  "answer": "Sorry, I could not find an answer. Please contact support.",
  "confidence": 0.41
}
```

12.3 Creating an FAQ (Admin)

```
POST /api/faqs
Authorization: Bearer <ADMIN_API_KEY>
Content-Type: application/json

{
  "question": "How do I change my email address?",
  "answer": "Go to Profile > Contact details and edit your email.",
  "category": "Account",
  "tags": ["email", "profile"]
}
```

A successful request returns **201 Created** with the stored record, including its identifier and timestamps. A missing question or answer returns **400 Bad Request**.

13. AI and Semantic Search

Semantic search matches questions by meaning rather than relying only on exact keywords. This is the central AI component of the project.

13.1 Embeddings

Text can be represented as numerical vectors called embeddings. Similar questions tend to have similar vector representations. For example, a stored FAQ might ask “How do I change my password?” while a user asks “I forgot my password; how can I create a new one?”. Although the words differ, their embeddings are close to each other, so semantic matching can identify the relationship.

13.2 Similarity Calculation

The similarity between the query vector A and a stored vector B is commonly measured using cosine similarity, defined as the dot product of the vectors divided by the product of their lengths: $\cos(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$. The value increases as the two vectors point in more similar directions, and the FAQ with the highest score is the best candidate.

13.3 Relevance Threshold and Fallback

A relevance threshold avoids returning unrelated answers. If the highest score is below the threshold, the API returns a fallback message. The threshold (for example 0.75) is stored in configuration and tuned by testing the system with real and unrelated questions: too high a value rejects valid questions, and too low a value returns wrong answers.

13.4 Implementation Sketch

```
function cosine(a, b) {
  let dot = 0, na = 0, nb = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
    na += a[i] * a[i];
    nb += b[i] * b[i];
  }
  return dot / (Math.sqrt(na) * Math.sqrt(nb));
}

async function answer(question) {
  const queryVec = await embed(question); // embedding service
  const faqs = await Faq.find({ isActive: true });
  let best = null, bestScore = 0;
  for (const faq of faqs) {
    const score = cosine(queryVec, faq.embedding);
    if (score > bestScore) { best = faq; bestScore = score; }
  }
  if (!best || bestScore < THRESHOLD) {
    return { matched: false, answer: FALLBACK, confidence: bestScore };
  }
  return { matched: true, answer: best.answer, confidence: bestScore };
}
```

13.5 The /api/ask Route

```
router.post("/ask", askLimiter, async (req, res, next) => {
  try {
    const { question } = req.body;
    if (typeof question !== "string" ||
        question.trim().length < 3 || question.length > 500) {
      return res.status(400).json({
        success: false,
        error: { code: "INVALID_QUESTION",
                  message: "Provide a question of 3 to 500 characters." }
      });
    }
    const result = await answer(question.trim());
    res.json({ success: true, question: question.trim(), ...result });
  } catch (err) { next(err); }
});
```

13.6 Improving Retrieval

The retrieval quality can be improved by embedding the question together with its tags, by combining keyword and semantic scores (hybrid search), and by reviewing low-confidence questions to add missing FAQ entries.

14. Authentication and Security

Administrative FAQ operations must be protected so that only authorised users can change the knowledge base. API keys, bearer tokens, sessions or another suitable authentication method can be used. In this project a bearer token or API key is checked by a middleware function before any create, update or delete operation.

```
const crypto = require("crypto");

function safeEqual(a, b) {
  const x = Buffer.from(a), y = Buffer.from(b);
  return x.length === y.length && crypto.timingSafeEqual(x, y);
}

function requireAdmin(req, res, next) {
  const header = req.get("Authorization") || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : "";
  if (!token || !safeEqual(token, process.env.ADMIN_API_KEY)) {
    return res.status(401).json({
      success: false,
      error: { code: "UNAUTHORIZED",
        message: "Valid admin credentials are required." }
    });
  }
  next();
}
```

The security measures applied in the project are summarised in Table 14.1.

Table 14.1: Security measures

Measure	Description
Authentication	Admin endpoints require a valid API key or bearer token.
Input validation	Missing, malformed or oversized fields are rejected with 400.
Rate limiting	Limits requests per client to reduce abuse and control AI-service cost.
Secure credential storage	Secrets are kept in environment variables or a secret manager, never in source code.
HTTPS	All traffic is encrypted in production.
Safe logging	Logs avoid storing tokens, secrets and personal data.

15. Error Handling

Reliable APIs provide predictable errors. Appropriate HTTP status codes and JSON error objects help client applications respond correctly. A central error-handling middleware catches exceptions from all routes so that internal details are logged but never exposed to the client.

Table 15.1: HTTP status codes used by the API

Status	Meaning	Typical use
200	Successful request	Listing FAQs, fetching an FAQ, answering a question
201	Resource created	A new FAQ was stored
400	Invalid request	Missing field, wrong type or empty question
401	Unauthorized	Missing or invalid admin credentials
404	Resource not found	No FAQ exists with the given ID
429	Rate limit exceeded	Too many requests from the same client
500	Server error	Unexpected failure or unavailable AI service

Example error response:

```
{
  "success": false,
  "error": {
    "code": "FAQ_NOT_FOUND",
    "message": "No FAQ was found with the given ID."
  }
}
```

If the embedding service is unavailable, the API returns a 500-level error with a generic message, and the failure is recorded in the server log for investigation.

16. Testing Strategy

Testing verifies that every endpoint behaves correctly for valid and invalid requests. Postman is used for manual API testing, while automated unit and integration tests (for example with Jest and Supertest) can be added later. The testing approach has three levels:

- **Unit testing:** individual functions such as the cosine similarity calculation and input validation.

- **Integration testing:** complete requests through the routes, middleware and database.
- **Manual API testing:** Postman collections that exercise each endpoint with valid, invalid and unauthorised requests.

The main scenarios covered by the test plan are listed below.

- Health endpoint test
- FAQ creation, update and deletion tests
- Valid AI question and unknown question
- Empty question
- Unauthorised request
- Rate-limit test

For semantic search, a set of paraphrased questions is prepared for each stored FAQ. The percentage of paraphrases that return the correct FAQ gives a practical measure of retrieval quality and helps in choosing the relevance threshold.

17. Test Cases and Expected Results

The following cases form the project test report. The Actual Result and Status columns are completed after execution.

Table 17.1: Test cases

ID	Input	Expected Result	Actual Result / Status
TC01	GET /	200 status	
TC02	Valid FAQ POST	201 Created	
TC03	Missing field	400 error	
TC04	Existing FAQ ID	200 response	
TC05	Invalid ID	404 error	
TC06	Valid /api/ask	Relevant answer	
TC07	Unknown question	Fallback response	
TC08	Unauthorized admin request	401 error	
TC09	Empty question in /api/ask	400 error	
TC10	Repeated /api/ask above the limit	429 error	

18. Advantages and Applications

The API-based design makes the assistant reusable. One backend can serve multiple client applications that follow the API contract.

18.1 Advantages

- Fast FAQ retrieval
- Natural-language questions
- Centralised knowledge management
- Reusable across applications
- Easy web and mobile integration
- Reduced repetitive support work

18.2 Applications

Potential applications include educational institutions (admission and examination queries), customer-support portals, internal help desks, service information systems, product support portals and public-information websites.

19. Future Enhancements

The system can be extended in several directions:

- **Multilingual support** so that users can ask and receive answers in their own language.
- **Voice input** through speech-to-text integration.
- **Conversation history** to handle follow-up questions.
- **Admin dashboard** for managing FAQs through a graphical interface.
- **Analytics** to identify the most common and unanswered questions.
- **Feedback system** so users can rate answers and improve the knowledge base.
- **Hybrid search** combining keyword and semantic search.
- **Cloud deployment** for scalable access by larger applications.

20. Conclusion

The Development of an AI FAQ Assistant API demonstrates how REST API development and AI-assisted information retrieval can be combined into a practical software system. The project provides structured endpoints for FAQ management and a question-answer interface for users.

The API-first approach makes the solution reusable and provides a foundation for future improvements such as multilingual interaction, voice support, analytics, authentication enhancements and cloud deployment. The AI-augmented development approach showed that AI tools can speed up routine work, provided the developer reviews and tests every result.

References

1. Node.js Documentation. <https://nodejs.org/docs>
2. Express.js Documentation. <https://expressjs.com>
3. MongoDB Documentation. <https://www.mongodb.com/docs>

4. Postman Documentation. <https://learning.postman.com>
5. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
6. MDN Web Docs: HTTP overview and status codes. <https://developer.mozilla.org>
7. General documentation on natural-language processing, text embeddings and cosine similarity.